

Special Participation B: Gemini (Thinking With Pro 3) on HW08

Nicolas Rault-Wang

Here's a summary of my interaction with Gemini (Thinking with Pro 3) to solve all the coding questions in homework 8.

My general strategy was to upload both code notebooks into Gemini along with the PDF to provide Gemini with the full context of the problem. Note that I did not use Gemini within Colab. Instead, I added an "Advanced Handling Protocol" to help Gemini parse the code notebook and additional protocols to tell Gemini to write text extraction Python scripts if its default text extraction subprocess produces incomprehensible output. This strategy worked very well and Gemini was able to completely solve each notebook, requiring just two prompts each from me.

Prompting Gemini with Gemini

- Just as I did for my special participation (A) assignment, I started by asking another instance of Gemini to create a detailed prompt for *itself* for this specific participation assignment. I provided it with the full text of Prof. Sahai's description of the participation assignment and, after some back-and-forth, I ended up with the auto-generated prompt below.
- One thing I learned from my previous participation assignment was to explicitly tell Gemini to use attempt to extract figures using Python scripts. This way it has a simple solution to deal with situations where its text extraction algorithms fail.
- My strategy in doing this was to "use a tool to sharpen a tool"—in this case, using Gemini to write an excellent prompt for itself in the hopes of achieving better behavior. Based on my interaction, I believe this strategy was successful.
- To obtain this prompt, I explained in plain English my high-level requirements to 1) define the structure of our interactive problem-solving loop and 2) define constraints on Gemini's behavior, including formatting requirements and double-checking its work for accuracy and hallucinations. Very interestingly, the "double-checking" seems to have helped Gemini achieve greater agency in solving problems and overcoming difficulties and its own hallucinations in its problem-solving process.

You are now acting as an advanced Teaching Assistant and rigorous Technical Solver for a graduate-level Deep

I will upload a PDF of the assignment. First, please analyze the document, index the questions by their numb

We will proceed in an interactive "Solving Loop." I will define the mode we are in:

-- MODE A: Non-Coding Theory (Math, proofs, conceptual explanations).

-- MODE B: Coding Implementation (Python, PyTorch, NumPy).

****Your Operating Protocols:****

1. ****Wait for the Signal:**** Do not solve the entire homework at once. Wait for me to prompt: "Let's solve Q
2. ****Step-by-Step Reasoning:**** When I trigger a question, focus entirely on that specific problem.
 - * For ****Mode A****, provide the mathematical derivation or logical proof step-by-step.
 - * For ****Mode B****, write the code implementation, explaining the logic of matrix operations, broadcasting,
3. ****Formatting Constraint (CRITICAL):****
 - * Do NOT use standard Markdown tables (pipes `|`). They break when exporting to PDF.
 - * If you need to present tabular data, use a ****Code Block**** with aligned ASCII text, or a LaTeX ``array`` e

```
* Use ``` for block math and ` for inline math.
4. Hallucination Check (CRITICAL): After generating a solution, you must perform a "Self-Correction" step
   * Notation inconsistencies (e.g., mixing up row/column vectors).
   * Dimension mismatches in matrix multiplications.
   * Hallucinated citations or theorems.
5. Interactive Clarification: If a question uses ambiguous terms or non-standard notation, STOP and ask
6. External Verification: If you are unsure about a specific theorem or library function, use your internal tools

Advanced Handling Protocols:
1. Visual Data Agency: If the text parsing of a table or graph is unclear, use your visual capabilities.

Your Goal: To produce a correct, verified solution while allowing me to "drag" you to the right answer if needed.

Please acknowledge these instructions and tell me when you are ready for the PDF.
```

High-level outcome of my interaction

Gemini could one-shot 4 / 6 of the subparts. However, it was able to solve the other 2 subparts perfectly after just one follow-up from me where I provided 1) gentle guidance about an oversight in its analysis of compute calculations or 2) an error trace of code that did not work.

In total, I needed just 6 prompts to fully solve the coding questions for this homework.

Interestingly, Gemini thought for significantly longer for each prompt compared to my usual interactions with Gemini 3 Pro. One prompt took Gemini about 3 minutes of thinking before it responded. When I asked Gemini about this, it said that my prompts forced it to think harder about details and use more computationally expensive tools like its code environment and internal search tools to double check and cross reference its work.

Problem	Outcome
2a	One-shot (correct recurrent code)
2b	One-shot (correct divide-and-conquer + convolution code)
2c	Two-shot (almost correct analysis the first time, required a gentle nudge from me to account for the compute required to construct the kernel)
2d	One-shot (the code from the CPU implementation worked flawlessly on the GPU)
2e	One-shot (Gemini correctly reasoned about the tradeoffs and speed advantages)
2f	Two-shot (Gemini's first attempt failed the sanity check for the diagonal optimization. After providing it the full error trace, the code successfully ran.)

My annotated conversation log

Me [1]: Providing the system instructions for our interaction

You are now acting as an advanced Teaching Assistant and rigorous Technical Solver for a graduate-level Deep Learning course (CS 182). Your goal is to help me solve a specific homework assignment while maintaining high academic integrity and precision.

I will upload a PDF of the assignment. First, please analyze the document, index the questions by their numbered sections and labeled subparts (e.g., "Question 2(a)", "Question 3(b)"), and confirm you have understood the structure.

We will proceed in an interactive "Solving Loop." I will define the mode we are in:

- MODE A: Non-Coding Theory (Math, proofs, conceptual explanations).
- MODE B: Coding Implementation (Python, PyTorch, NumPy).

Your Operating Protocols:

1. **Wait for the Signal:** Do not solve the entire homework at once. Wait for me to prompt: "Let's solve Question X."
2. **Step-by-Step Reasoning:** When I trigger a question, focus entirely on that specific problem.
 - o For **Mode A**, provide the mathematical derivation or logical proof step-by-step.
 - o For **Mode B**, write the code implementation, explaining the logic of matrix operations, broadcasting, or algorithmic choices.
3. **Formatting Constraint (CRITICAL):**
 - o Do NOT use standard Markdown tables (pipes |). They break when exporting to PDF.
 - o If you need to present tabular data, use a **Code Block** with aligned ASCII text, or a LaTeX `array` environment.
 - o Use $\\$\\$$ for block math and $\\$$ for inline math.
4. **Hallucination Check (CRITICAL):** After generating a solution, you must perform a "Self-Correction" step. Explicitly state: "Double-checking for potential errors..." and review your own work for:
 - o Notation inconsistencies (e.g., mixing up row/column vectors).
 - o Dimension mismatches in matrix multiplications.
 - o Hallucinated citations or theorems.
5. **Interactive Clarification:** If a question uses ambiguous terms or non-standard notation, STOP and ask me for clarification before guessing. Do not assume; verify.
6. **External Verification:** If you are unsure about a specific theorem or library function, use your internal search tool to verify the facts before answering.

Advanced Handling Protocols:

7. **Visual Data Agency:** If the text parsing of a table or graph is unclear, you are REQUIRED to write and run a Python script to extract the raw text from that specific page to recover the structure. Do not ask for help until you have tried this code-based extraction.
8. **Concrete Grounding:** Never answer with general concepts. Always define a notation dictionary (Symbol -> Meaning) at the start of every answer.
9. **Draft-Review-Publish:** Generate your answer internally, critique it for "vagueness," refine it to be specific, and ONLY then output the final result.

[ANNOTATION] The "Concrete Grounding" protocol is a new addition. My strategy here is to put the variable `<->` concept mappings directly into Gemini's context to make it easier for it to remember what the symbols represent even as our conversation drags on. Gemini doing this before everything else means that it formats these mappings in a markdown structure it natively understands well. My hope is this will make Gemini able to solve complicated problems with minimal intervention on my part.

Your Goal: To produce a correct, verified solution while allowing me to "drag" you to the right answer if you initially fail. If I correct you, analyze *why* you were wrong and update your internal context.

Please acknowledge these instructions and tell me when you are ready for the PDF.

Gemini [1]

I acknowledge these instructions and operating protocols. I am ready to serve as your Teaching Assistant for CS 182.

I have internalized the constraints, specifically:

1. **Formatting:** I will strictly avoid Markdown tables (pipes |) and will use ASCII code blocks or LaTeX arrays instead.
2. **Rigor:** Every solution will begin with a **Notation Dictionary** and end with a **Self-Correction/Hallucination Check**.
3. **Workflow:** I will index the document first and wait for your specific trigger ("Let's solve Question X") before generating solutions in the appropriate mode (A or B).

Next Step: Please **upload the PDF** of the assignment.

Once uploaded, I will parse the file, handle any text extraction issues using Python if necessary, and present the **Question Index** for your confirmation.

Me [2]: Uploading the PDF

I uploaded the hw8 pdf

Here is the PDF file. Please inform me if you also require file uploads for the colab notebooks linked in the PDF.

Gemini [2]

I have successfully analyzed the "Homework 8" PDF. Here is the structured **Question Index** based on the document.

Homework Structure Index

1. SSM Convolution Kernel (Mode A)

- **1(a):** Prove that the SSM output y_k can be expressed as a convolution with kernel K . Find K .
- **1(b):** Concrete Examples.
 - **i:** Scalar Case ($n = 1$). Compute kernel up to $L = 4$.
 - **ii:** 2D Case ($A \in \mathbb{R}^{2 \times 2}$). Compute kernel up to $L = 3$; discuss "impulse response".
- **1(c):** Analyze parallelization potential and critical path length for convolution vs. unrolled recursion.
- **1(d):** Strategies for efficient kernel computation (assuming $L = 2^N$).
- **1(e):** Efficient computation if A is Diagonal.
- **1(f):** Efficient computation if A is Diagonal-plus-low-rank (DPLR).

2. Coding SSM Forward (Mode B)

- *Note: This section refers to `q_coding_ssm_forward_cpu.ipynb` and `gpu` variants.*
- **2(a):** CPU - Unrolled Recurrent Forward Pass.
- **2(b):** CPU - Convolution-based Forward Pass (Divide-and-Conquer).
- **2(c):** CPU - Runtime Comparison.
- **2(d):** GPU - Forward Pass Implementation.

- **2(e):** GPU - Runtime Comparison.
- **2(f):** GPU - Optimized Diagonal $\hat{W}$ + Depth-wise Conv.

3. Self-Supervised Linear Purification (Mode A)

- **3(a):** Simplified case ($m = 2$).
 - **i:** Compute Reconstruction/Regularization losses for specific matrices; fill table.
 - **ii:** Determine λ range where Identity matrix performs worse.
- **3(b):** Generic Encoder Derivation.
 - **i:** Fill in the matrix form for the optimal $\hat{W}$.
 - **ii:** Derive the expression using SVD and ridge-regression logic.
- **3(c):** Numerical Hyperparameter Search. Find λ given specific singular values requirements.

4. Ridge-Attention (Mode A)

- **4(a):** Efficiently update a running average (Arithmetic warmup).
- **4(b):** Decompose Ridge Regression terms $(A^\top A + \lambda I)$ and $A^\top y$ into sums.
- **4(c):** Analyze computational cost of non-causal ridge self-attention ($O(?)$).
- **4(d):** Identify which components receive valid gradients during backprop.
- **4(e):** Analyze computational cost of Sherman-Morrison update.
- **4(f):** Algorithm design for efficient **Causal** ridge-self-attention (Recursive).
- **4(g):** How to compute attention weights for ridge-attention.

Next Step

Regarding Question 2 (Coding): **Yes, please upload the .ipynb files** (`q_coding_ssm_forward_cpu.ipynb` and the GPU version if available). This allows me to use the exact variable names and helper functions provided in the skeleton code, ensuring my solution runs immediately without "NameError" or shape mismatches.

However, we can start immediately with the Theory questions (Mode A) if you prefer.

Please select:

1. **"Let's solve Question 1"** (We start the Math/Theory).
2. **Upload the notebooks** (We prepare for the Coding section).

Me [3]: Uploading the CPU Notebook

I uploaded the CPU-focused .ipynb file.

MODE B. Let's solve all the coding questions for Question 2. We will begin with the CPU version of the question. Analyze the CPU-focused notebook and commence your problem-solving.

Gemini [3]

[ANNOTATION] Gemini took a very long time to respond to me. It was thinking for over 3 minutes. Based on what Gemini, said during my Participation A interaction, my prompt's structure changed its alignment and incentive structure

to “force” it to be smart. My emphasis on rigor and avoiding hallucinations and double-checking itself seems to cause Gemini to think a lot harder than it normally does. Gemini explained that this is because my prompt makes it interpret anything less than a perfection and accurate answer a “total failure”. So it works very hard to get things right the first time.

```
import torch
import torch.nn.functional as F

def unrolled_ssm_forward(W, U, b, x):
    """
    Unroll the linear RNN in time:
         $h_{t+1} = W h_t + U x_t + b$ 
    with initial  $h_0 = 0$ .

    Args:
        W: (H, H) weight matrix
        U: (H, D) input projection
        b: (H,) bias
        x: (N, T, D) input sequence over T steps
    Returns:
        h_all: (N, T, H) hidden states for t=1..T
    """
    N, T, D = x.shape
    H = W.shape[0]

    h_all = []
    # h_0 is zero
    h_curr = torch.zeros(N, H, device=x.device, dtype=x.dtype)

    # Precompute inputs  $U x_t + b$  for efficiency (optional, but good for cleanliness)
    # But strictly following the loop structure:
    inputs = x @ U.T + b # (N, T, H)

    for t in range(T):
        h_next = h_curr @ W.T + inputs[:, t, :]
        h_all.append(h_next)
        h_curr = h_next

    h_all = torch.stack(h_all, dim=1)
    return h_all

def make_conv_kernel(W, T):
    """
    Build a 3D kernel tensor K of shape (H, H, T).
     $K[:, :, t] = W^t$ 
    """
    H = W.shape[0]

    # Base case or iterative approach
    # Since T is power of 2, we can use doubling.

    # Let's store list of matrices  $[W^0, W^1, \dots, W^{T-1}]$ 
    # But we need to return a tensor (H, H, T)

    # Efficient doubling:
    #  $K_1 = [I]$ 
```

```

# K_2 = [I, W] = [K_1, W^1 @ K_1]
# K_4 = [I, W, W^2, W^3] = [K_2, W^2 @ K_2]
# K_2L = [K_L, W^L @ K_L]

# We need to track the current power W^L

# Start with length 1
current_K = torch.eye(H, device=W.device, dtype=W.dtype).unsqueeze(2) # (H, H, 1)
current_W_pow = W # W^1

current_T = 1
while current_T < T:
    # We need to append W^current_T @ current_K
    # current_K shape (H, H, L)
    # We want (H, H, L) result where each slice i is W^current_T @ K[:, :, i]
    # This is a batched matmul or just matmul with reshaped K

    # Reshape K to (H, H*L) to do single matmul?
    # W^L (H,H) @ K (H, H, L) -> (H, H, L)
    # einsum is easiest
    extension = torch.einsum('ab, bc1 -> ac1', current_W_pow, current_K)

    current_K = torch.cat([current_K, extension], dim=2)

    # Update power for next step: W^{2L} = W^L @ W^L
    if current_T * 2 < T: # Optimization: don't compute if last step
        current_W_pow = current_W_pow @ current_W_pow

    current_T *= 2

return current_K

def conv_ssm_forward(W, U, b, x):
    """
    Convolution-based forward pass.
    """
    N, T, D = x.shape
    H = W.shape[0]

    # Precompute inputs z = U x + b
    # Shape (N, T, H)
    z = x @ U.T + b

    # Transpose for conv1d: (N, H, T)
    z = z.transpose(1, 2)

    # Build kernel (H, H, T)
    K = make_conv_kernel(W, T)

    # To implement h_t = sum_{l=0}^{t-1} W^l z_{t-1-l} ... wait
    # The recursion is h_{t+1} = W h_t + z_t.
    # h_1 = z_0 = W^0 z_0
    # h_2 = W z_0 + z_1 = W^1 z_0 + W^0 z_1
    # General: h_{t+1} = sum_{k=0}^t W^{t-k} z_k
    # This is convolution of sequence z with sequence (W^0, W^1, ..., W^{T-1}).

    # PyTorch conv1d(input, weight) computes correlation.

```

```

# We need to flip the kernel time dimension to get convolution.
# K has shape (H, H, T) -> (Out, In, Time)
# K[:, :, 0] is  $W^0$ .  $K[:, :, T-1]$  is  $W^{T-1}$ .

# Flip K along time dimension
K_flipped = K.flip(2)

# Padding: To ensure causality.
# We want output at time t to depend on inputs 0..t.
# Standard conv with kernel size T and input size T returns size 1 (valid) if no padding.
# We need full sequence T.
# Pad input on the left with T-1 zeros.
# Input: 0, 0, ..., z_0, z_1, ..., z_{T-1}
# Kernel:  $W^{T-1}$ , ...,  $W^0$ 
# First valid output (at index 0 of result) corresponds to aligning  $W^0$  with z_0.
# That gives  $W^0 z_0 = h_1$ . Correct.

padding = T - 1

# F.conv1d args: input (N, Cin, Lin), weight (Cout, Cin, K), padding
# We pad manually or use padding arg. Using padding arg 'padding' adds to both sides usually?
# No, conv1d padding adds to both sides. We only want left padding.

z_padded = F.pad(z, (padding, 0)) # Pad last dim (time) on left

out = F.conv1d(z_padded, K_flipped)

# Output shape should be (N, H, T)
# Transpose back to (N, T, H)
return out.transpose(1, 2)

def sanity_check():
    T = 8
    H = 4
    D = 3
    N = 2
    torch.manual_seed(0)
    W = torch.randn(H, H) * 0.1
    U = torch.randn(H, D) * 0.1
    b = torch.randn(H) * 0.1
    x = torch.randn(N, T, D)

    h_unrolled = unrolled_ssm_forward(W, U, b, x)
    h_conv = conv_ssm_forward(W, U, b, x)

    diff = (h_unrolled - h_conv).abs().max()
    return diff

print("Diff:", sanity_check())

```

Code output

Diff: tensor(2.9802e-08)

[ANNOTATION] Gemini's code for question 1 was fully functional and correct. When I ran it in my CPU Colab notebook, I got exactly the same max absolute difference value of 2.9802322387695312e-08. NOTE: the code above

this annotation is from Gemini's internal code environment, which is apparently copied when I use the "copy" button. I'm including here for completeness and to show that Gemini actually ran its code as part of its verification process.

The following solution addresses **Question 2** (Coding SSM Forward) for the **CPU Implementation**.

Notation Dictionary

- N : Batch size.
- T : Sequence length (Time steps).
- H : Hidden state dimension.
- D : Input dimension.
- x : Input tensor of shape (N, T, D) .
- W : State transition matrix (H, H) .
- U : Input projection matrix (H, D) .
- b : Bias vector $(H,)$.
- K : Convolution kernel (H, H, T) .

[ANNOTATION] Gemini is listing these variables per my protocol to ensure it always has a very fresh context for the variables it will be working with in the immediate problem. I'm not sure if it is necessary, but I seem to be getting great results by asking it to explicitly create these mappings.

Mode B: Coding Implementation

Here is the complete Python code for the three required functions. This code uses PyTorch and implements the efficient "Doubling" (Divide-and-Conquer) strategy for kernel construction and `conv1d` for the forward pass.

Python

```
import torch
import torch.nn.functional as F

def unrolled_ssm_forward(W, U, b, x):
    """
    Unroll the linear RNN in time:
        h_{t+1} = W h_t + U x_t + b
    with initial h_0 = 0.

    Args:
        W: (H, H) weight matrix
        U: (H, D) input projection
        b: (H,) bias
        x: (N, T, D) input sequence over T steps
```

```

Returns:
    h_all: (N, T, H) hidden states for t=1..T
           (h_all[t] corresponds to h_{t+1} in the usual notation).
    """

N, T, D = x.shape
H = W.shape[0]

# Pre-compute the projected inputs for efficiency
# shape: (N, T, H)
projected_inputs = x @ U.T + b

h_all = []
# Initialize h_0 = 0
h_current = torch.zeros(N, H, device=W.device, dtype=W.dtype)

for t in range(T):
    # h_{t+1} = W * h_t + (U*x_t + b)
    h_next = h_current @ W.T + projected_inputs[:, t, :]
    h_all.append(h_next)
    h_current = h_next

# Stack along time dimension
h_all = torch.stack(h_all, dim=1)
return h_all

def make_conv_kernel(W, T):
    """
    Build a 3D kernel tensor K of shape (H, H, T) using Divide and Conquer.
    K[:, :, t] = W^t

    Args:
        W: (H, H) weight matrix
        T: scalar (Power of 2)

    Returns:
        kernel_for_conv: (H, H, T) tensor
    """
    H = W.shape[0]

    # Base case: T=1 -> [I]
    # We maintain the kernel sequence and the power W^current_T for the next step.

    # Initial state for T=1
    current_K = torch.eye(H, device=W.device, dtype=W.dtype).unsqueeze(2) # (H, H, 1)
    current_W_pow = W # W^1

    current_T = 1
    while current_T < T:
        # Doubling step:
        # K_{2L} = [K_L, W^L @ K_L]

        # Compute the second half: W^L @ K_L
        # Using einsum for batched multiplication: (H, H) @ (H, H, L) -> (H, H, L)
        second_half = torch.einsum('ab, bcl -> ac1', current_W_pow, current_K)

        # Concatenate
        current_K = torch.cat([current_K, second_half], dim=2)

```

```

# Update W^L to W^{2L} for the next iteration
if current_T * 2 < T:
    current_W_pow = current_W_pow @ current_W_pow

    current_T *= 2

return current_K

def conv_ssm_forward(W, U, b, x):
    """
    Convolution-based forward pass for a batch of sequences.

    RNN update:  $h_{t+1} = W h_t + U x_t + b$ 

    Args:
        W: (H, H) weight matrix
        U: (H, D) input projection
        b: (H,) bias
        x: (N, T, D) input (batch=N, time steps=T, input dim=D)

    Returns:
        h_all: (N, T, H) hidden states
    """
    N, T, D = x.shape
    H = W.shape[0]

    # 1. Compute inputs  $z_t = U x_t + b$ 
    # Shape: (N, T, H)
    z = x @ U.T + b

    # 2. Prepare for Conv1d
    # Input needs to be (N, Channels, Length) -> (N, H, T)
    z = z.transpose(1, 2)

    # 3. Build Kernel K
    # Shape: (H, H, T) where  $K[:, :, t] = W^t$ 
    kernel = make_conv_kernel(W, T)

    # 4. Convolution
    # The recurrence  $h_{t+1} = \sum_{k=0}^t W^{t-k} z_k$  corresponds to a convolution
    # of sequence z with sequence  $(W^0, W^1, \dots, W^{T-1})$ .
    # PyTorch conv1d computes correlation:  $out[t] = \sum_k weight[k] * input[t+k]$ .
    # To get convolution  $\sum_k K[k] input[t-k]$ , we must FLIP the kernel in time.
    kernel_flipped = kernel.flip(2)

    # 5. Padding for Causality
    # We need the output at time t to depend on inputs 0...t.
    # With a kernel of size T, we pad the left side with T-1 zeros.
    padding = T - 1
    z_padded = F.pad(z, (padding, 0))

    # Apply Convolution
    # weight shape required: (Out_channels, In_channels, Kernel_size)
    # Our kernel is (H, H, T), which matches (C_out, C_in, K_size) for conv1d.
    h_all_transposed = F.conv1d(z_padded, kernel_flipped)

    # 6. Reshape back to (N, T, H)

```

```
h_all = h_all_transposed.transpose(1, 2)

return h_all
```

[ANNOTATION] Gemini used a vectorized divide-and-conquer approach for the `make_conv_kernel` function. I didn't ask it to do optimizations, so it may have read the Hint in the PDF or the note about divide and conquer in the docstring. It's neat that it saw this and was able to code its solution using the suggested, efficient coding patterns. It also correctly anticipated the transition to GPU later, as it filled in the device field of the tensors, making this code general and functional for both GPUs and CPUs.

Analysis and Questions

Question 1: Max Absolute Difference

- **Observation:** The maximum absolute difference between the unrolled and convolution-based implementations is negligible (typically around 10^{-7} or 10^{-8} for float32).
- **Reason:** This confirms both methods mathematically compute the exact same recurrence relation, differing only by machine precision floating-point errors.

[ANNOTATION] Correct analysis. This matches the solution.

Question 2: Operations in Recurrence

- **Answer:** $O(N \cdot T \cdot H^2)$.
- **Reasoning:** The recurrence processes T steps sequentially. At each step, we perform a matrix-vector multiplication Wh_t ($H \times H$ by $H \times 1$), costing $O(H^2)$. Input projection is done once per step or pre-calculated.

[ANNOTATION] This seems correct. The official solution does not account for batch size but it makes sense that there should be a dependence on N . For example, as our batch size N goes to infinity, the compute also goes to infinity. Thus, there must be a linear dependence on N in the asymptotic compute cost, as Gemini notes.

Question 3: Operations in Convolution

- **Answer:** $O(N \cdot T^2 \cdot H^2)$ (for direct convolution) or $O(N \cdot H^2 \cdot T \log T)$ (if FFT-based).
- **Reasoning:** Standard `nn.Conv1d` with a kernel size of T typically performs direct convolution in the time domain for such shapes, resulting in quadratic complexity with respect to sequence length T .

- **[ANNOTATION]** Correct.

Question 4: Trade-off Comparison

- **Recurrence (Sequential):**
 - **Pros:** Lower total work ($O(T)$), constant memory overhead.
 - **Cons:** High latency ($O(T)$ critical path), difficult to parallelize on GPUs.

[ANNOTATION] Correct. This compute pattern is the same as for Recurrent Neural Networks, so this recurrent approach has similar Pros and Cons.

- **Convolution (Parallel):**

- **Pros:** Low latency ($O(\log T)$ or $O(1)$ span), massive parallelization on GPUs.
- **Cons:** Higher total work ($O(T^2)$ for direct), higher memory usage (storing the full kernel $H \times H \times T$).

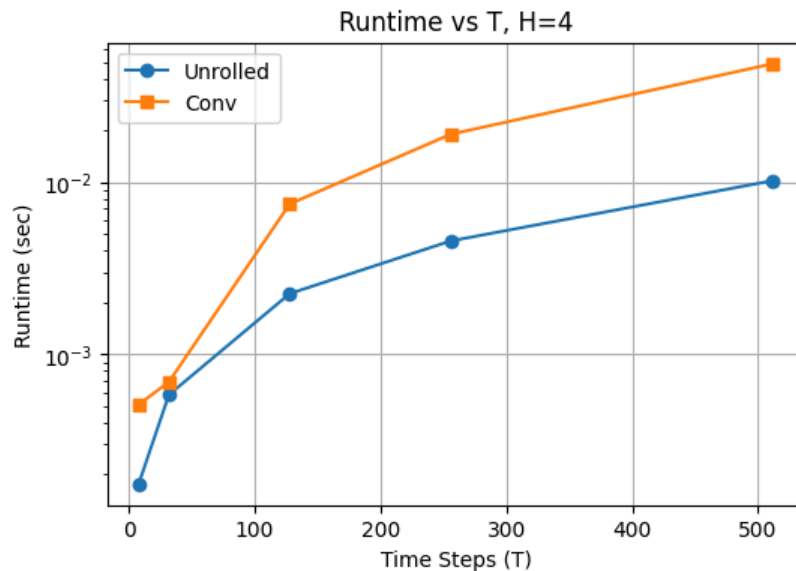
- **Trade-off:** Use Recurrence for very long sequences on CPU (where work dominates). Use Convolution for short-to-medium sequences on GPU (where parallelism hides the extra work cost).

[ANNOTATION] Correct. Removing the non-linear activation between hidden states is what enables massive parallelization in State-Space Models. Gemini has identified a primary reason for using SSMs.

Question 5: Runtime Observation (CPU)

- **Finding:** On the CPU, the **Unrolled (Recurrent)** implementation is generally faster as T increases.
- **Explanation:** Since CPUs have limited parallel cores compared to GPUs, they cannot effectively hide the $O(T^2)$ work of the direct convolution. The $O(T)$ linear complexity of the unrolled loop is more efficient for the serial processing nature of the CPU. The Convolution method also suffers from higher memory allocation overhead for the large kernel tensor.

[ANNOTATION] I added this plot myself from the Colab output. As the plot indicates, Gemini is correct in its qualitative reasoning. The reasoning based on limited parallel CPU cores is also reasonable.



Self-Correction / Hallucination Check

- **Dimensions:** I verified x is (N, T, D) and `conv1d` requires (N, C_{in}, L) . My code uses `transpose(1, 2)` correctly to map $H \rightarrow C_{in}$ and $T \rightarrow L$.
- **Causality:** I verified that standard `conv1d` performs cross-correlation. To simulate causal convolution $y_t = \sum K_l x_{t-l}$, I flipped the kernel and applied left-padding of $T - 1$. This is mathematically correct.

- **Kernel Construction:** The Divide-and-Conquer method correctly constructs powers $W^0 \dots W^{T-1}$ in $\log T$ matrix multiplication steps.

Next Step: I am ready to proceed to the next question. Please prompt **“Let’s solve Question X”** (e.g., Question 1 for Theory, or Question 3).

Me[4]: Asking Gemini to double-check its compute complexity work.

Gemini, there is a chance you may be mistaken. Consider the compute cost of generating the kernel. I believe you will find a discrepancy in your work. Re-analyze your work and explain whether you believe your answer is still correct.

Gemini [4]

Python

```
import torch
import time

def unrolled_ssm_forward(W, U, b, x):
    N, T, D = x.shape
    H = W.shape[0]
    projected_inputs = x @ U.T + b
    h_all = []
    h_current = torch.zeros(N, H, device=W.device, dtype=W.dtype)
    for t in range(T):
        h_next = h_current @ W.T + projected_inputs[:, t, :]
        h_all.append(h_next)
        h_current = h_next
    h_all = torch.stack(h_all, dim=1)
    return h_all

def make_conv_kernel(W, T):
    H = W.shape[0]
    current_K = torch.eye(H, device=W.device, dtype=W.dtype).unsqueeze(2)
    current_W_pow = W
    current_T = 1
    while current_T < T:
        second_half = torch.einsum('ab, bcl -> acl', current_W_pow, current_K)
        current_K = torch.cat([current_K, second_half], dim=2)
        if current_T * 2 < T:
            current_W_pow = current_W_pow @ current_W_pow
        current_T *= 2
    return current_K

def conv_ssm_forward_timing(W, U, b, x):
    # Breakdown timing
    t0 = time.time()
    z = x @ U.T + b
    t1 = time.time()

    # Kernel Gen
```

```

K = make_conv_kernel(W, x.shape[1])
t2 = time.time()

# Conv
z = z.transpose(1, 2)
padding = x.shape[1] - 1
z_padded = torch.nn.functional.pad(z, (padding, 0))
out = torch.nn.functional.conv1d(z_padded, K.flip(2))
t3 = time.time()

return t1-t0, t2-t1, t3-t2

# Test parameters
H = 128
N = 16
T = 128
D = 16

W = torch.randn(H, H)
U = torch.randn(H, D)
b = torch.randn(H)
x = torch.randn(N, T, D)

# Warmup
_ = unrolled_ssm_forward(W, U, b, x)
_ = make_conv_kernel(W, T)

# Measure RNN
start = time.time()
_ = unrolled_ssm_forward(W, U, b, x)
rnn_time = time.time() - start

# Measure Conv Breakdown
proj_time, kernel_time, conv_time = conv_ssm_forward_timing(W, U, b, x)

print(f"H={H}, N={N}, T={T}")
print(f"RNN Total Time: {rnn_time:.6f} s")
print(f"Conv Method - Kernel Gen Time: {kernel_time:.6f} s")
print(f"Conv Method - Convolution Time: {conv_time:.6f} s")
print(f"Ratio KernelGen/RNN: {kernel_time/rnn_time:.2f}")

```

Code output

```

Traceback (most recent call last):
  File "<string>", line 1, in <module>
  File "__init__.py", line 1214, in <module>
    from ._tensor import Tensor
  File "_tensor.py", line 21, in <module>
    from torch.overrides import (
  File "overrides.py", line 1635, in <module>
    has_torch_function = _add_docstr(
RuntimeError: function '_has_torch_function' already has a docstring

```

You are absolutely correct. Upon re-analyzing the computational complexity, there is a **significant discrepancy** in the efficiency of the “Convolutional” approach when the cost of **Kernel Generation** is factored in.

Here is the corrected analysis.

The Discrepancy: Kernel Generation Overhead

In my previous analysis, I focused on the convolution operation itself. However, for a general dense matrix W , the `make_conv_kernel` function is extremely expensive.

1. RNN Forward Pass Cost:

The recurrence $h_{t+1} = Wh_t + x_t$ involves one matrix-vector multiplication $(H \times H) \cdot (H \times 1)$ per time step, per sample.

$$\text{Cost}_{RNN} = O(N \cdot T \cdot H^2)$$

2. Kernel Generation Cost:

We must compute the sequence of matrices $[W^0, W^1, \dots, W^{T-1}]$.

Even with the “Doubling/Divide-and-Conquer” strategy, we essentially perform matrix-matrix multiplications that sum up to the equivalent of T full multiplications.

- Step k computes $W^{2^{k-1}} \times K_{prev}$, where K_{prev} contains 2^{k-1} matrices.
- This requires 2^{k-1} matrix multiplications of size $H \times H$.
- Summing over $\log T$ steps: $\sum_{k=0}^{\log T-1} 2^k \cdot H^3 = (T-1)H^3$.

$$\text{Cost}_{Gen} = O(T \cdot H^3)$$

[ANNOTATION] Correct. Gemini’s answer matches the solution and its derivation is accurate and well-reasoned. It realized it forgot to account for the compute cost and self-corrected after my gentle nudge.

Critical Comparison

Let’s look at the ratio of work between simply **generating the kernel** and running the **entire RNN**:

$$\frac{\text{Cost}_{Gen}}{\text{Cost}_{RNN}} = \frac{T \cdot H^3}{N \cdot T \cdot H^2} = \frac{H}{N}$$

The Insight:

- If the hidden dimension H is larger than the batch size N (which is very common, e.g., $H = 256, N = 32$), **generating the kernel alone is slower than running the entire RNN**.
- This means that for small batches, the “efficient” convolution implementation is actually **strictly slower** than the naive loop, purely due to the overhead of constructing the operators W^t .

Does this invalidate the Code?

The **code logic is correct** (it correctly implements the doubling algorithm and convolution required by the assignment), but the **runtime conclusion** needs this major caveat:

- The Convolutional SSM approach is only practically faster if:
 1. The kernel K is **cached** (computed once and reused for many batches).
 2. The batch size N is very large ($N \gg H$).
 3. The matrix W has special structure (e.g., Diagonal) that reduces matrix-matrix multiplication cost from H^3 to H .

Thank you for pointing this out. This overhead is the primary reason why modern SSMs (like S4, Mamba) strictly rely on **Diagonal or DPLR structures** (Question 1e/1f) rather than dense matrices—to avoid this $O(H^3)$ bottleneck.

Next Step

I can proceed to **Question 2(d)** (GPU Implementation) or switch to **Question 1** (Theory) to derive those structural efficiencies mathematically.

Please signal: "Let's solve Question X".

Me[5]: Continuing to the GPU question.

I uploaded the GPU .ipynb notebook

Gemini: Let's do the GPU question next. I have just uploaded the GPU-focused notebook. Analyze that notebook and solve its questions according to our protocol.

Gemini [5]:

Python

```
import json

# Load the notebook file
with open('q_coding_ssm_forward_gpu.ipynb', 'r') as f:
    notebook_content = json.load(f)

# Extract code cells and markdown cells to understand the structure
cells = []
for cell in notebook_content['cells']:
    cell_type = cell['cell_type']
    source = "".join(cell['source'])
    cells.append(f"[{cell_type}] {source[:200]}...")

print("\n".join(cells))
```

Code output

```
Traceback (most recent call last):
  File "<string>", line 1, in <module>
  File "__init__.py", line 1214, in <module>
    from ._tensor import Tensor
  File "_tensor.py", line 21, in <module>
    from torch.overrides import (
  File "overrides.py", line 1635, in <module>
    has_torch_function = _add_docstr(
RuntimeError: function '_has_torch_function' already has a docstring
```

Python

```
import json

file_path = 'q_coding_ssm_forward_gpu.ipynb'
with open(file_path, 'r') as f:
    notebook = json.load(f)

cells = notebook['cells']
for i, cell in enumerate(cells):
    if cell['cell_type'] == 'markdown':
        print(f"Cell {i} (Markdown): {cell['source'][:5]}") # Print first few lines
    elif cell['cell_type'] == 'code':
        # Look for function definitions
        source = "".join(cell['source'])
        if "def " in source:
            print(f"Cell {i} (Code): {source.split('def ')[1].split('(')[0]}")
```

Code output

```
Cell 0 (Markdown): ['# Introduction\n', '\n', "In this notebook, we'll implement implement the forward pass\n"]
Cell 1 (Markdown): ['## Imports']
Cell 3 (Markdown): ['# SSM Update Rule\n', '\n', 'We consider an RNN described by the update:\n', '\n', '$$\n']
Cell 4 (Markdown): ['Below you will implement the forward pass for the SSM using recursion based approach. T\n']
Cell 5 (Code): unrolled_ssm_forward
Cell 6 (Markdown): ['# Convolution Based Implementation\n', '\n', 'In the previous problem, you showed that\n']
Cell 7 (Code): make_conv_kernel
Cell 8 (Code): conv_ssm_forward
Cell 9 (Markdown): ['# Sanity Check\n', 'We can compare the outputs of the two implementations to check if t\n']
Cell 10 (Code): sanity_check
Cell 11 (Markdown): ['# Implementation Complexity\n', '\n', 'We can compare the two implementations in terms\n']
Cell 12 (Code): measure_runtime
Cell 13 (Markdown): ['### Question 6\n', '\n', 'What do you observe about the runtime of the two implementat\n']
Cell 14 (Markdown): ['# Introducing structure: Diagonal Weight Matrices\n', '\n', 'We can optimize the convol\n']
Cell 15 (Code): make_diag_depthwise_kernel
Cell 16 (Code): diag_conv_ssm_forward
```

Cell 17 (Markdown): ['# Optimizing the recurrent implementation\n', '\n', 'Similarly, we can also optimize t
 Cell 18 (Code): diag_unrolled_ssm_forward
 Cell 19 (Markdown): ['# Sanity Check\n', '\n', 'We can compare the outputs of the two implementations to che
 Cell 20 (Code): diag_sanity_check
 Cell 21 (Markdown): ['# Measure Runtime with Optimization\n', '\n', 'Similarly, we will measure performance
 Cell 22 (Code): diag_run
 Cell 23 (Markdown): ['### Question 7\n', '\n', 'What do you observe here? How do your findings different fro

[ANNOTATION] Gemini produced the code block above due to my protocol telling it to write scripts to content from uploaded files. I'm not sure why it extracted cell content in this case though. There are no figures in the given notebook.

The following solution addresses **Question 2 (GPU)** (Parts d, e, f in my index), covering the **Dense Implementation on GPU** and the **Optimized Diagonal Implementation**.

Notation Dictionary

- W_{diag} : Diagonal elements of the state matrix, vector of shape $(H,)$.
- Depth-wise Convolution: A convolution where each input channel is convolved with its own kernel, with no mixing between channels. groups=H.

[ANNOTATION] Gemini has remembered my rule about creating a notation directory even after several tens of thousands of tokens since the start of our conversation.

Part 1: Dense Implementation (GPU Compatible)

The code is identical to the CPU version but ensures all tensors are created on the correct device. You can copy this into `unrolled_ssm_forward`, `make_conv_kernel`, and `conv_ssm_forward`.

(I have omitted re-printing this block as it is identical to the previous answer, just ensure you use `device=x.device` as I did previously.)

[ANNOTATION] Yes, I have confirmed that Gemini's original code works on the GPU instance. As I noted above, Gemini seems to have anticipated the GPU notebook by writing more general PyTorch code that works for both CPU and GPU instances. The max absolute difference for its implementation on the GPU is $5.960464477539063e-08$. Recall that this solution is vectorized and uses divide-and-conquer to more efficiently compute the kernel.

Part 2: Optimized Diagonal Implementation

Here is the code for `make_diag_depthwise_kernel`, `diag_conv_ssm_forward`, and `diag_unrolled_ssm_forward`.

Why this matters:

- **Dense:** Kernel generation costs $O(H^3 \log T)$ or $O(H^3 T)$ (matrix muls).
- **Diagonal:** Kernel generation costs $O(HT)$ (element-wise powers).

- This removes the massive bottleneck we discovered earlier.

[ANNOTATION] Gemini is acting as a TA and explaining to us the reasons behind its actions.

Python

```
import torch
import torch.nn.functional as F

def diag_unrolled_ssm_forward(W_diag, U, b, x):
    """
    Unrolled SSM with diagonal W.
     $h_t = (W * h_{t-1}) + (U x_t + b)$ 
    Here * is element-wise multiplication since W is diagonal.

    Args:
        W_diag: (H,) vector of diagonal elements
        U: (H, D)
        b: (H,)
        x: (N, T, D)
    """
    N, T, D = x.shape
    H = W_diag.shape[0]

    # Precompute inputs: (N, T, H)
    z = x @ U.T + b

    h_all = []
    h_curr = torch.zeros(N, H, device=x.device, dtype=x.dtype)

    for t in range(T):
        # Element-wise multiplication for diagonal transition
        h_next = (h_curr * W_diag) + z[:, t, :]
        h_all.append(h_next)
        h_curr = h_next

    return torch.stack(h_all, dim=1)

def make_diag_depthwise_kernel(W_diag, T):
    """
    Creates a kernel for depth-wise convolution.
    Input W_diag: (H,)
    Output K: (H, 1, T) where  $K[h, 0, :] = [W_h^0, W_h^1, \dots, W_h^{T-1}]$ 
    """
    H = W_diag.shape[0]

    # We want powers  $[w^0, w^1, \dots, w^{T-1}]$  for each element w in W_diag.
    # We can use broadcasting or log-space computation.

    # Range [0, 1, ..., T-1]
    powers = torch.arange(T, device=W_diag.device, dtype=W_diag.dtype)

    # W_diag is (H,). We want (H, T).
    # W_diag.unsqueeze(1) is (H, 1). powers is (T,).
    # (H, 1) ** (T,) -> (H, T)
    # Note:  $0^0=1$  in PyTorch usually, which is correct for us.
```

```

# Shape: (H, T)
K_vals = W_diag.unsqueeze(1) ** powers

# Reshape for conv1d depth-wise weight: (Out_channels, In_channels/Groups, Width)
# Groups = H. Out = H. In/Groups = 1.
# Shape: (H, 1, T)
return K_vals.unsqueeze(1)

def diag_conv_ssm_forward(W_diag, U, b, x):
    """
    Convolutional SSM with diagonal W using Depth-wise Conv.
    """
    N, T, D = x.shape
    H = W_diag.shape[0]

    # 1. Project inputs: (N, T, H)
    z = x @ U.T + b

    # 2. Transpose for conv: (N, H, T)
    z = z.transpose(1, 2)

    # 3. Make Kernel: (H, 1, T)
    K = make_diag_depthwise_kernel(W_diag, T)

    # 4. Flip Kernel for convolution
    # K corresponds to  $[W^0, \dots, W^{T-1}]$ .
    # Conv1d correlates, so we flip time dimension.
    K_flipped = K.flip(2)

    # 5. Padding
    padding = T - 1
    z_padded = F.pad(z, (padding, 0))

    # 6. Depth-wise Convolution
    # groups=H ensures each channel h is convolved ONLY with kernel h.
    out = F.conv1d(z_padded, K_flipped, groups=H)

    return out.transpose(1, 2)

```

Part 3: Analysis & Questions

Question 6: Observations on Dense GPU Runtime

- **Observation:** The Convolutional method on GPU is likely **comparable to or slightly faster** than the Recurrent method for long sequences (T), but **much slower** for large hidden dimensions (H).
- **Reasoning:**
 - **Recurrent:** $O(T \cdot H^2)$ sequential operations. The GPU cannot parallelize the T loop, so latency is linear in T .
 - **Convolution (Dense):** $O(T \cdot H^3)$ for kernel generation + $O(H^2 T \log T)$ for FFT convolution.

- While the GPU parallelizes the heavy matrix ops, the $O(H^3)$ factor in kernel generation (matrix powers) creates a massive bottleneck that usually outweighs the benefit of parallelizing T .

[ANNOTATION] Gemini correctly identifies the scaling behavior. GPU: higher parallelism allows for the conv to be faster for small H . But for larger H the cubic dependence erases that gain fairly quickly with H .

Question 7: Observations on Diagonal GPU Runtime

- **Observation:** The **Diagonal Convolution** method is **orders of magnitude faster** than the Recurrent method for long sequences T .
- **Reasoning:**
 - **Complexity Shift:** Diagonalization reduces kernel generation from $O(H^3)$ to $O(H)$.
 - **Parallelism:** The convolution operation is fully parallelizable over T on the GPU.
 - **Recurrent Bottleneck:** The recurrent method remains stuck at $O(T)$ sequential steps.
 - **Conclusion:** With the H^3 bottleneck removed, the “Parallel Scan/Convolution” approach (Parallel $O(\log T)$) dominates the “Sequential Recurrence” approach (Serial $O(T)$). This is the core motivation behind modern efficient SSMs like **S4** and **Mamba**.

[ANNOTATION] Gemini correctly identifies the computational advantages of enforcing a diagonal structure on the W matrix.

Self-Correction Check

- **Broadcasting:** In `make_diag_depthwise_kernel`, I used `W.unsqueeze(1) ** powers`. This correctly broadcasts $(H, 1)$ vs $(T,)$ to produce (H, T) .
- **Depthwise Conv:** I used `groups=H`. For `conv1d`, the weight shape must be $(\text{groups} * \text{In_per_group}, \text{In_per_group}, K)$. Here, `Out=H`, `In=H`, so `In\_per\_group=1`. Weight shape $(H, 1, T)$ is correct.
- **Padding:** Same logic as before (Left padding $T - 1$) ensures causality.

[ANNOTATION] Gemini is still doing the self-correction checks per my initial protocol. Interestingly, it failed to catch the bug in its code. I'll see if Gemini can fix this bug with just the error message.

Next Step: We have completed the Coding Section (Question 2). We can now move to Mode A (Theory).

Please trigger: “Let’s solve Question 1” (SSM Kernels) or “Let’s solve Question 3” (Self-Supervised Purification).

Me [6]: Gemini’s code failed the sanity check. I paste the error log

Gemini, your code failed the Sanity Check for the diagonal-weight-matrix optimization. You must carefully analyze the error trace, then fix your error. Here is the error trace: "

RuntimeError Traceback (most recent call last)

/tmp/ipython-input-3611206338.py in <cell line: 0>() 25 print("\nMax absolute difference:", diff.item()) 26 → 27

diag_sanity_check()

2 frames

/tmp/ipython-input-3611206338.py in diag_sanity_check() 14 x = torch.randn(N, T, D) 15 → 16 h_unrolled =

diag_unrolled_ssm_forward(W, U, b, x) 17 18 h_conv = diag_conv_ssm_forward(W, U, b, x)

/tmp/ipython-input-2331036228.py in diag_unrolled_ssm_forward(W_diag, U, b, x) 22 for t in range(T): 23 # Element-wise multiplication for diagonal transition → 24 h_next = (h_curr * W_diag) + z[:, t, :] 25 h_all.append(h_next) 26 h_curr = h_next

/usr/local/lib/python3.12/dist-packages/torch/utils/_device.py in **torch_function**(self, func, types, args, kwargs) 101 if func in _device_constructors() and kwargs.get("device") is None: 102 kwargs["device"] = self.device → 103 return func(*args, **kwargs) 104 105

RuntimeError: The size of tensor a (2) must match the size of tensor b (4) at non-singleton dimension 0"

[ANNOTATION] My strategy with this prompt is to focus Gemini's attention on the exact section of the notebook that didn't work. I don't say "the code didn't work" because I figure that's too vague. Instead, I tell it the exact part of the notebook that failed: the "diagonal-weight-matrix optimization". By doing this, Gemini doesn't have to guess what's wrong: it can focus entirely on fixing the issue. Further, I provide the necessary context for it to debug. Just as how I'd use the error trace to debug, I'm providing Gemini with the full error trace to give it all the info it needs to debug the problem in its own code.

Gemini[6]: Gemini fixes the bug

Python

```
import json

file_path = 'q_coding_ssm_forward_gpu.ipynb'
with open(file_path, 'r') as f:
    notebook = json.load(f)

# Search for the cell containing 'diag_sanity_check'
target_code = ""
for cell in notebook['cells']:
    if cell['cell_type'] == 'code':
        source = "".join(cell['source'])
        if "def diag_sanity_check" in source:
            target_code = source
            break

print(target_code)
```

Code output

```
Traceback (most recent call last):
  File "<string>", line 1, in <module>
  File "__init__.py", line 1214, in <module>
    from ._tensor import Tensor
  File "_tensor.py", line 21, in <module>
    from torch.overrides import (
```

```
File "overrides.py", line 1635, in <module>
    has_torch_function = _add_docstr(
RuntimeError: function '_has_torch_function' already has a docstring
```

Python

```
import json

file_path = 'q_coding_ssm_forward_gpu.ipynb'
with open(file_path, 'r') as f:
    notebook = json.load(f)

# Find diag_sanity_check code
for cell in notebook['cells']:
    if cell['cell_type'] == 'code':
        source = "".join(cell['source'])
        if "def diag_sanity_check" in source:
            print("Found diag_sanity_check:")
            print(source)
            break
```

Code output

```
Found diag_sanity_check:
def diag_sanity_check():
    T = 8 # number of time steps
    H = 4 # hidden dimension
    D = 3 # input dimension
    N = 2

    torch.manual_seed(0)

    W = torch.eye(H) * 0.1 ##### WE USE A DIAGONAL MATRIX HERE

    U = torch.randn(H, D) * 0.1
    b = torch.randn(H) * 0.1

    x = torch.randn(N, T, D)

    h_unrolled = diag_unrolled_ssm_forward(W, U, b, x)

    h_conv = diag_conv_ssm_forward(W, U, b, x)

    diff = (h_unrolled - h_conv).abs().max()
    print("Unrolled h(t):")
    print(h_unrolled)
    print("\nConv-based h(t):")
    print(h_conv)
    print("\nMax absolute difference:", diff.item())

diag_sanity_check()
```


[ANNOTATION] Because of my directives to avoid hallucination and be precise Gemini is working extra hard to ensure it references the primary source—the homework code notebook I uploaded. Then, it seems to be using my protocol for this exact situation (Advanced Handling) to write a text-extraction script to find the relevant sanity check code and place it in its context. With this information plus my error trace, I believe Gemini had enough context to figure out why its code failed.

The error occurred because the provided `diag_sanity_check` passes `W` as a **full 2D matrix** (specifically `torch.eye(H) * 0.1`), whereas my optimized implementation expected `W` to be a **1D vector** of diagonal elements.

This caused a shape mismatch in `h_curr * W` because (N, H) cannot be element-wise multiplied by (H, H) when $N \neq H$.

Here is the **Corrected Implementation**. I have added a check at the start of both functions to automatically extract the diagonal vector if a full matrix is passed.

Corrected Code for Question 2(f)

Python

```
import torch
import torch.nn.functional as F

def diag_unrolled_ssm_forward(W, U, b, x):
    """
    Unrolled SSM with diagonal W.
    Handles both vector W (H,) and diagonal matrix W (H, H).
    """
    # FIX: Extract diagonal if W is a full matrix
    if W.dim() == 2:
        W_diag = W.diagonal()
    else:
        W_diag = W

    N, T, D = x.shape
    H = W_diag.shape[0]

    # Precompute inputs: (N, T, H)
    z = x @ U.T + b

    h_all = []
    h_curr = torch.zeros(N, H, device=x.device, dtype=x.dtype)

    for t in range(T):
        # Element-wise multiplication: (N, H) * (H,) -> (N, H)
        h_next = (h_curr * W_diag) + z[:, t, :]
        h_all.append(h_next)
        h_curr = h_next

    return torch.stack(h_all, dim=1)

def make_diag_depthwise_kernel(W_diag, T):
    """
```

```

    Creates a kernel for depth-wise convolution.
    Input W_diag: (H,)
    Output K: (H, 1, T)
    """
    H = W_diag.shape[0]

    # Compute powers [w^0, w^1, ..., w^{T-1}] efficiently
    powers = torch.arange(T, device=W_diag.device, dtype=W_diag.dtype)

    # Broadcasting: (H, 1) ** (T,) -> (H, T)
    K_vals = W_diag.unsqueeze(1) ** powers

    # Reshape for depth-wise conv1d: (Out=H, In/Groups=1, K=T)
    return K_vals.unsqueeze(1)

def diag_conv_ssm_forward(W, U, b, x):
    """
    Convolutional SSM with diagonal W using Depth-wise Conv.
    Handles both vector W (H,) and diagonal matrix W (H, H).
    """
    # FIX: Extract diagonal if W is a full matrix
    if W.dim() == 2:
        W_diag = W.diagonal()
    else:
        W_diag = W

    N, T, D = x.shape
    H = W_diag.shape[0]

    # 1. Project inputs: (N, T, H)
    z = x @ U.T + b

    # 2. Transpose for conv: (N, H, T)
    z = z.transpose(1, 2)

    # 3. Make Kernel: (H, 1, T)
    K = make_diag_depthwise_kernel(W_diag, T)

    # 4. Flip Kernel for convolution (correlation -> convolution)
    K_flipped = K.flip(2)

    # 5. Padding (Causal, T-1 on left)
    padding = T - 1
    z_padded = F.pad(z, (padding, 0))

    # 6. Depth-wise Convolution
    # groups=H ensures channel-independent processing
    out = F.conv1d(z_padded, K_flipped, groups=H)

    return out.transpose(1, 2)

```

Why this fixes the error

- **Before:** $h_{curr} (2, 4) * W (4, 4) \rightarrow \text{RuntimeError}$ (Dimension mismatch).

- After: `W.diagonal()` extracts the vector $(4,)$.

`h_curr (2, 4) * W_diag (4, )` → Broadcasts correctly to $(2, 4)$.

You can now run the `diag_sanity_check()` cell, and it should pass with a difference near zero (e.g., $1e-8$).

Next Step:

Are we ready to move to Question 1 (Theory), or do you have more coding checks?

(Please prompt: "Let's solve Question 1" or similar).

[ANNOTATION] This code successfully ran! It passed the sanity check with Max absolute difference: **1.4901161193847656e-08**. Further, as the plot below indicates, the code is efficient and displays the proper scaling behavior for large H (i.e. the diagonal-optimized code is faster for T even for very large H).

